

SkipReduce: (Interconnection) Network Sparsity to Accelerate Distributed Machine Learning

Hans Kasan*

Korea Advanced Institute of Science and Technology
Daejeon, Republic of Korea
hanskasan@kaist.ac.kr

Jungwook Choi

Hanyang University
Seoul, Republic of Korea
choij@hanyang.ac.kr

Dennis Abts

NVIDIA
Santa Clara, CA, USA
dabts@nvidia.com

John Kim

Korea Advanced Institute of Science and Technology
Daejeon, Republic of Korea
jjk12@kaist.edu

Abstract

The interconnection network is a critical component for building scalable systems, as its communication bandwidth directly impacts the collective communication performance of distributed training. In this work, we exploit interconnection network sparsity (or communication sparsity) to address challenges of communication performance and scalability. In particular, we identify how gradients (or packets) during communication can be randomly skipped with minimal impact on accuracy. However, skipping gradients in fine granularity (or individually) results in a loss of gradient information without improving communication performance, due to the synchronous nature of collective communication. Thus, we propose *coarse-grained* skipping where gradient slices are skipped, which enables skipping of some AllReduce steps to accelerate communication. In particular, we propose *SkipReduce* collective communication that intentionally skips random gradients during AllReduce. However, a naive implementation of SkipReduce can degrade accuracy by repeatedly skipping gradients from the same node, which introduces bias. To mitigate this accuracy loss, we show how randomizing the skipped gradient slices improves training accuracy with negligible additional runtime. We also observe that not all layers have similar communication sparsity and propose applying SkipReduce *selectively* where only the sparse layers (or gradients) are skipped to minimize the accuracy impact of SkipReduce. Compared to prior work on communication acceleration, SkipReduce can be seamlessly integrated into existing collective communication libraries with minimal overhead. We implement SkipReduce on top of NCCL's ring-based AllReduce algorithm. Our results show that this method accelerates collective communication while preserving final training accuracy. Compared to baseline AllReduce, SkipReduce provides up to a $1.58\times$ speedup in time-to-accuracy. Beyond this performance gain in data parallelism, this work also

discusses the broader implications of SkipReduce, including its application to other parallelism strategies and logical topologies, as well as its benefits as a model regularizer.

ACM Reference Format:

Hans Kasan, Dennis Abts, Jungwook Choi, and John Kim. 2025. SkipReduce: (Interconnection) Network Sparsity to Accelerate Distributed Machine Learning. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3725843.3756092>

1 Introduction

The complexity of Deep Neural Networks (DNNs) has been steadily increasing, both in terms of model [31] and dataset size [6] in pursuit of higher accuracy. The increase in model parameters in turn intensifies computational demand, necessitating more workers (e.g., GPUs) to accelerate DNN training. To cope with the increasing computational demand, large-scale high-performance computing (HPC) systems featuring dedicated, high-performance interconnects have been widely used. Prominent examples include NVIDIA Selene [7], Google TPUv4 [19], and Tesla Dojo [48]. On these systems, workloads are distributed across multiple workers or nodes, allowing parallel learners to cooperate in a process known as distributed training. The computational workloads are distributed by either splitting the training data or the DNN model itself, often referred to as data and model-parallel training, respectively [31, 50]. Unfortunately, the speedup in training time scales sub-linearly with the number of workers [32, 43] as distributed training incurs communication overhead. For data parallelism, an AllReduce collective communication is required after each gradient update to synchronize the model across all workers. Due to the synchronous nature of distributed training, each worker must idle until communication is complete before starting the next training iteration. This exposed communication time directly reduces worker utilization, wasting valuable computing resources.

This work addresses the communication challenges from AllReduce collective operations across large-scale training systems. AllReduce collectives consist of two phases: a Reduce-Scatter phase, where gradients from all workers are reduced, and an AllGather phase, where the final reduced gradients are distributed to all other workers. For a system with N workers using ring-based AllReduce, each phase requires $(N - 1)$ steps – resulting in $2(N - 1)$ steps. In this work, we explore how the number of steps in AllReduce can be

*Part of this work was conducted during an internship at NVIDIA Research.



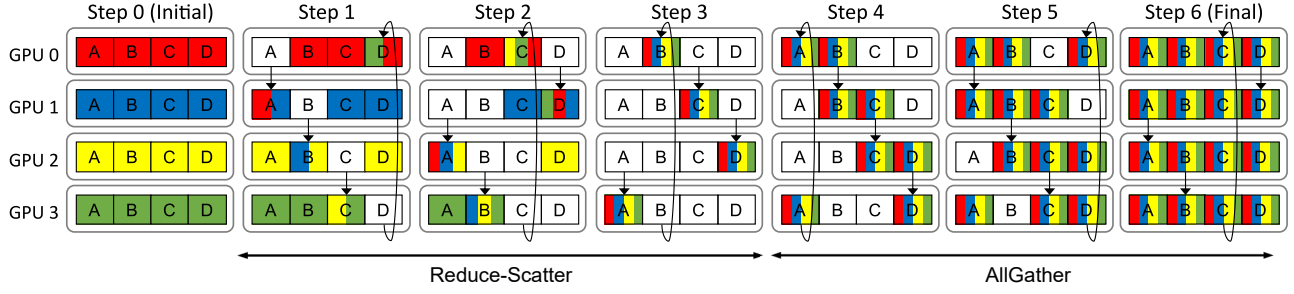


Figure 1: Baseline Ring AllReduce with 4 workers (e.g., GPUs) consisting of 3 steps for Reduce-Scatter and 3 additional steps for AllGather. Each box represents a message *slice* with the letters indicating different slices. We assume a single unidirectional ring and the arrows indicate the slice movements.

reduced by “skipping” gradients through the proposed SkipReduce algorithm. Similar to weights in DNNs, the gradients also tend to be “sparse” and have values that are normally distributed with a mean near zero – thus, not all gradients are strictly necessary. In this work, we propose a simple technique to randomly skip gradients during the collective communication. Prior works have looked into methods for accelerating collective communication, primarily by reducing the message size through various compression techniques. A common approach is magnitude-based compression, where only the top- k gradients (those with the largest magnitudes) are communicated in a sparse coordinate format (COO). These gradients are sent along with their indices, which introduces additional communication overhead [3, 12, 25, 38, 41, 42]. Another major challenge of the top- k approach is the ‘fill-in’ issue. Because the indices of top- k gradients differ across workers, the total size of the aggregated sparse message grows with each step of the AllReduce operation. Furthermore, since top- k approaches communicate using the sparse COO format, they cannot be easily integrated into existing collective communication libraries (e.g., NCCL [1]). PowerSGD proposed gradient compression using low-rank approximation [51]. As the gradients are still communicated as dense messages, this approach does not suffer from the fill-in issue and can be integrated into collective communication libraries. However, the compression overhead can increasingly be a bottleneck to performance (i.e., communication time) as the channel bandwidth grows.

In this work, we explore the observation that gradients can be randomly skipped (similar to using Dropout [45] to avoid overfitting) using our proposed SkipReduce collective communication to reduce communication overhead of AllReduce and exploit (interconnection) network sparsity.¹ SkipReduce reduces communication time by “skipping” some of the AllReduce steps, which is functionally equivalent to randomly skipping a fraction of the gradients. However, a naive implementation of SkipReduce is problematic since skipping the same communication steps results in the same slices being skipped. As a result, we propose *Random SkipReduce* where the workers being skipped are continuously varied. We further identify that sparsity is not uniform across the layers of a

neural network. Moreover, we find that important gradients, defined by their magnitude, are often concentrated in only a few parts of the model. To maintain accuracy, we implement SkipReduce to *selectively* skip gradients from layers that are less sensitive to this process. Our evaluations demonstrate that on a communication-intensive VGG-19 training, the training iteration time can be reduced by up to 21% on an 8-GPU system when 50% of gradients are skipped, while having minimal impact on training accuracy. In particular, the contributions of this work include the following:

- We observe that gradients can be randomly skipped in AllReduce collective communication with minimal impact on overall training accuracy by exploiting the (interconnection) network sparsity.
- We exploit coarse-grained network sparsity to propose a practical implementation of gradient skipping through *SkipReduce* where some communication steps of AllReduce are skipped, which can be implemented with no overhead on existing collective communication libraries.
- We observe how naive implementation of SkipReduce can hurt accuracy since the same gradient slices are always skipped, and show how we can improve accuracy by randomizing the slices to be skipped with negligible overhead.
- We observe how different parts of the neural network exhibit varying levels of sparsity, and propose to apply SkipReduce *selectively* to preserve accuracy.

2 Background

2.1 Deep Neural Network Training

Deep neural network (DNN) training is performed in two steps: forward and backward propagation. During forward propagation, the DNN is fed with an input x , which is propagated through each layer to produce the output y and to obtain the scalar loss $L(w_t)$. The loss is a function of w_t , the weight of the DNN at iteration t . During backward propagation, the information from the loss flows backward through each layer to compute the gradients. The weights are then updated in the direction of the gradients to minimize the loss. This optimization method is called gradient descent, as shown in Equation 1.

$$w_{t+1} = w_t - \alpha \nabla_{w_t} L(w_t) \quad (1)$$

¹In this work, network sparsity refers to interconnection network sparsity or the potential sparsity in the data (or packets) communicated among the workers (e.g., GPUs), not necessarily neural network sparsity.

Table 1: High-level comparison of various collective communication algorithms. ‘+’: good, ‘-’: poor.

Algorithm	Communi- cation	Pattern	Sparsity	Indexing Overhead	Preprocessing Overhead	Reconstruction Overhead	Error Accumulation
Baseline AllReduce	dense	regular	none	+	+	+	+
Top- k [25, 30, 42]	sparse	irregular	high	-	-	-	-
PowerSGD [51]	sparse	regular	high	+	-	-	-
Gradient Quantization [9]	dense	regular	none	+	-	-	+
SkipReduce (This Work)	sparse	regular	medium	+	+	+	+

where α is the scalar learning rate, which helps to adjust the step size of the weight updates.

Depending on the amount of data used for each weight update, gradient descent can be classified as deterministic or stochastic. *Deterministic* or *batch* gradient descent uses the entire training dataset, or the “batch”, for every single update. On the other hand, *stochastic* or *minibatch* gradient descent uses only a small, randomly sampled subset of the training data, which is often referred to as the “minibatch.” The size of the minibatch is a tunable parameter [14]. In this work, we train using stochastic gradient descent (SGD).

2.2 Distributed Data Parallel Training

Data parallel training [50] distributes the training dataset among a group of workers, where each worker trains an identical replica of the DNN model. As a result, each worker performs forward propagation with different input data, obtains different loss values, and computes different gradients after backward propagation. Before each weight update, the gradients are averaged across the workers using an AllReduce collective communication to synchronize the workers [12]. These averaged gradients are then used to update the weights on each worker’s model replica [6]. As a result, at the end of each training step, every worker holds an identical model. In this work, we focus on the commonly used Ring AllReduce [34], which operates by constructing one or more logical rings over the physical network topology. Figure 1 shows the step-by-step operation of the Ring AllReduce on a 4-GPU system. A Ring AllReduce with N workers completes in $2(N - 1)$ steps. Thus, for the 4-GPU example, the operation finishes in six steps. Initially, the local gradients on each GPU are first divided into *slices*, which are the fundamental unit of transfer. The number of slices is equal to the number of GPUs participating in the collective communication. In each step, every GPU sends one slice to its neighbor. After the first $(N - 1)$ steps, every GPU holds one slice that contains the reduced gradient information from all other GPUs – this is referred to as the Reduce-Scatter phase. The next phase, AllGather, circulates these fully reduced slices so that every GPU ends up with an identical copy of the reduced gradients. This phase also takes $(N - 1)$ steps to complete.

2.3 Related Works

A high-level qualitative comparison of previously proposed techniques to accelerate communication time in distributed training, in particular for data parallelism, is shown in Table 1. The table compares SkipReduce with prior works across several key characteristics. *Communication* specifies whether the whole data is sent

(dense) or only partially (sparse). *Pattern* indicates if the sparsity pattern is regular, which allows the message size to be reduced without converting the message format, while irregular sparsity pattern necessitates the usage of sparse message format, e.g., coordinate (COO) format. *Sparsity* indicates the sparsity level, i.e., the proportion of data skipped during communication. We also compare the associated cost or overhead of each algorithm. *Indexing Overhead* refers to the computation cost due to the conversion to sparse message format. *Preprocessing Overhead* is the computation overhead for preprocessing the message before transmission, such as message compression in top- k -based approaches [25, 30, 42] and PowerSGD [51], or quantization to lower-precision formats. *Reconstruction Overhead* is the computational cost of restoring a received message back to its original format, e.g., converting a sparse message back to dense message format (as in top- k -based methods) or dequantizing low-precision values back to a higher precision format (as in gradient quantization). Finally, *Error Accumulation* indicates if the method requires an extra memory space for error accumulation, e.g., to store unsent gradients in top- k -based approaches or to enable error feedback in PowerSGD by storing the difference between the computed and compressed gradients. Prior approaches introduce overheads as they often rely on sparse, irregular communication; use compression methods that add indexing, preprocessing, and reconstruction costs; or require extra memory to track compression errors. In comparison, SkipReduce relies on regular communication patterns already common in collective communication libraries with no additional computation or memory overhead, allowing it to be seamlessly integrated into widely used libraries.

Gradient compression approaches (e.g., DGC [30], gTopk [42], OkTopk [25]) leverage the observation that not all weights in the DNN models undergo significant updates, which results in the gradients being mostly sparse. Therefore, communicating all the values of the gradient is not necessary to achieve convergence [6]. These approaches propose selecting and sending only the most important gradients, using their magnitude as the metric for their importance. This is often called the top- k selection, where only the gradients whose magnitude falls within the top k percent are transmitted. The selected gradients are sent in the coordinate (COO) format, which pairs each value with its corresponding indices. The main drawback is that this indexing doubles the total message size [25, 38, 41, 42]. The communication time speedup of these sparsification approaches is simply $\frac{1}{2k} \times$, where k is the compression ratio, and the factor of 2 in the denominator accounts for the COO format overhead. Conversion to and from the COO format incurs

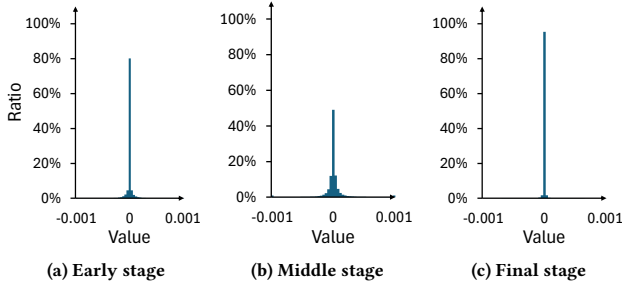


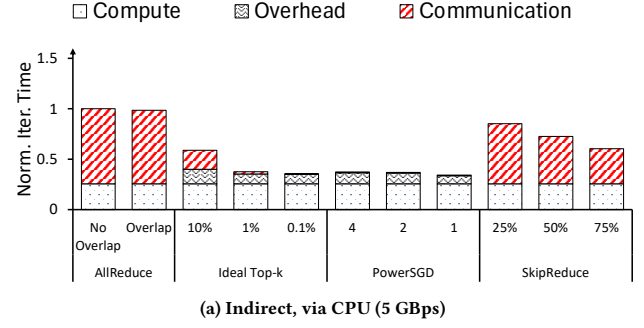
Figure 2: Distribution of the gradient values at three representative stages during VGG-19 training on CIFAR-100 dataset.

indexing and reconstruction overhead, respectively. Moreover, gradients not selected for transmission are accumulated locally with the gradients from subsequent iterations. This continues until their accumulated magnitude is large enough to be selected [25, 30, 46]. The primary drawback is that storing these residual gradients incurs a memory overhead equivalent to the full model size.

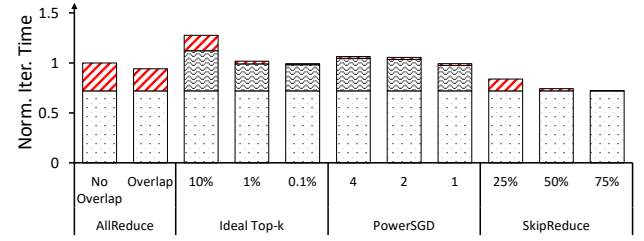
Prior works have explored various methods to minimize the preprocessing overhead of selecting important gradients. One approach is to use a static threshold to select the top- k gradients [47]. However, this approach suffers from significant error [11], as the optimal threshold changes every iteration [3]. Thus, most recent works utilize a dynamic threshold for top- k gradient selection. SparCML proposed selecting the top- k gradients locally at each worker and execute an AllGather operation to synchronize the gradients across all workers [38]. However, one key challenge is that the selected gradient indices often differ across workers. As these sparse messages are aggregated during communication, the total number of unique indices – and thus, the message size – grows, diminishing the benefits of sparsification. This problem is commonly known as the *fill-in* issue. To mitigate this issue, gTopk proposed to recalculate the top- k gradients at each AllReduce step to keep the message size constant [42] but suffers from high computation overhead. OkTopk proposes to execute additional AllReduce operations to ensure that the number of local top- k gradient values across the workers is approximately the same to balance the communication load [25].

Low-rank approximation is another method for gradient compression. A notable example is PowerSGD, which uses power iteration [51]. Instead of sending the full gradients, it only communicates the approximation, which is much smaller, thereby reducing the message size. The receiver then reconstructs the full gradient tensor using the received low-rank tensors. Its main benefit is that it avoids irregular communication patterns, making it compatible with existing collective communication libraries. However, this approach introduces computational overhead to create the low-rank approximation and requires memory overhead for error accumulation.

Gradient quantization is a method for reducing communication overhead by representing gradients with lower-precision numerical formats [3, 38]. This approach provides a theoretical speedup of $\frac{32}{b_q} \times$ over a 32-bit representation like FP32, where b_q is the bit-width of the quantized format. Prior works have attempted to represent



(a) Indirect, via CPU (5 GBps)



(b) Direct GPU-to-GPU (16 GBps)

Figure 3: Breakdown of VGG-19 iteration time for various collective communications on an 8-GPU system across different channel bandwidths. The iteration time is normalized to AllReduce (no overlap) for each respective channel bandwidth.

the gradients using fewer bits while showing minimal loss in accuracy. Gupta et al. demonstrate that training using 16-bit wide representation results in negligible degradation in image classification accuracy [17]. Aji et al. observe sufficient training accuracy on the MNIST [24] dataset using a 2-bit representation [3]. 1-bit representations have also been considered [40, 47] and the convergence guarantees of low-precision SGD have also been explored [4, 39]. It is important to note that gradient quantization is an orthogonal approach to gradient sparsification and compression approaches, as well as to SkipReduce.

Prior works have proposed in-network computation [15, 18, 20, 26, 27, 36] to accelerate communication by offloading the computation to the network. Software-based approaches have also been proposed to accelerate collective communications, e.g., novel logical topologies [18, 23, 28], logical topology synthesizer [55], and collective communication scheduler [37]. The resiliency of distributed data parallel training towards gradient dropping has also been exploited to reduce the tail latency of collective communications over lossy network transports, as demonstrated by MLT [52] and OPTIREDUCE [53]. These works are orthogonal to SkipReduce and can be employed together to accelerate communication.

3 Motivation

In this section, we describe the motivation for SkipReduce, which stems from the observation that gradients communicated during distributed training are often sparse. While prior works have also

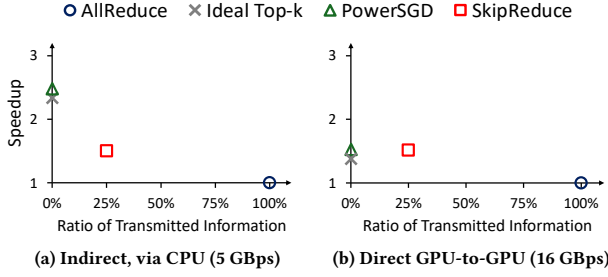


Figure 4: Tradeoff between performance (i.e., iteration time) and the amount of information communicated in LLaMA-3.2 1B training across different channel bandwidths. Communication-computation overlap is enabled. PowerSGD and ideal top- k transmit very little information (close to 0%) and achieve high speedup at low bandwidth (a), but speedup is reduced at higher bandwidth due to computation overhead (b).

exploited this observation, they typically rely on sorting and compressing the gradients. However, we show how the computation overhead for such approaches can be significant as the channel bandwidth increases. In comparison, the SkipReduce proposed in this work does not have such computation overhead and, because of its simplicity, can be easily implemented in collective communication libraries. In this work, we propose to exploit network sparsity to improve communication performance by skipping some communication steps in AllReduce, effectively dropping gradients in “bunches” – thus, resulting in a reduction of communication time. Skipping gradients is essentially equivalent to assuming that they are zeros, and since the gradients are sparse, assuming them as zeros is a reasonable estimation.

Prior works [3, 11, 25, 38, 42, 47] have shown that the gradients communicated during data parallel training are often narrowly distributed around zero or sparse. Figure 2 shows the gradient distribution over time throughout VGG-19 training. The gradient distribution differs across training stages – as the training progresses, the sparsity level decreases, which is shown by a wider distribution in Figure 2b, indicating a broader range of weights being updated. However, as the model converges, the gradients become increasingly sparse as shown in Figure 2c, reflecting fewer updates to the weights. In general, while the gradient distribution varies across different training stages, the gradients remain centered around zero throughout the entire training process, suggesting that data parallel training inherently exhibits high interconnection network sparsity.

While prior works have shown significant speedup in communication time, the benefits are typically evaluated on bandwidth-constrained systems (approximately 1 GBps bandwidth or lower [25, 29, 30, 38, 42, 51]). However, these approaches incur significant computation overhead on higher-bandwidth systems, which diminishes their benefit. Figure 3 provides the iteration time breakdown of VGG-19 [44] training with various collective communications on a system equipped with 8 A40 GPUs across different channel bandwidths.² We compare AllReduce with and without overlapping communication with computation, denoted as Overlap and

No Overlap, respectively. For all other collective communications, overlapping is always enabled. To adjust the bandwidth, we configure the NCCL_P2P_DISABLE environment variable. Disabling peer-to-peer (or GPU-to-GPU) communication forces the data to go through the host CPU, reducing the bandwidth to approximately 5 GBps. When peer-to-peer communication is enabled, a bandwidth of up to 16 GBps can be achieved. This value is lower than the theoretical maximum of PCIe 4.0 supported by A40 GPUs since the interconnect between CPU sockets becomes the performance bottleneck. At lower bandwidths, where the training time is dominated by communication time, the communication time savings significantly outweigh the computational overhead, enabling compression or approximate-based approaches to achieve a significant speedup. However, as bandwidth increases, the communication time decreases while the computation overhead remains constant. Eventually, the cost of executing compression or approximation dominates the communication time, diminishing the benefits of such techniques, as illustrated in Figure 3b.

To further illustrate the limitations of prior works on higher-bandwidth systems, Figure 4 presents the tradeoff between the ratio of transmitted information and communication time speedup. Ideally, communication should be both fast and with minimal amount of information loss, corresponding to the top-right region of the plots. Prior works either compress or approximate the gradients to aggressively reduce the message size, transmitting only a small fraction of information to achieve substantial speedups at low bandwidth. However, at higher bandwidths, these approaches suffer from the same level of information loss while offering diminished speedups, as their computation overhead becomes more prominent. In contrast, although SkipReduce provides relatively less speedup at lower bandwidths, it achieves comparable speedups to prior works at higher bandwidths with significantly less information loss. This advantage is further validated in the time-to-accuracy evaluations presented in Section 5.2.

4 SkipReduce Collective Communication

Leveraging the observation that gradients are sparse, we explore how training time can be reduced by skipping gradients and propose the SkipReduce collective communication. Unlike prior works, SkipReduce introduces *no* computation overheads (indexing, pre-processing and reconstruction overhead) and can be easily integrated into existing collective communication libraries.

4.1 Preliminaries

In this work, we differentiate between *skipping* and *dropping* of gradients. Skipping gradients refers to where the values of the gradients are ignored and effectively replaced with zeros. In comparison, dropping gradients refers to where the gradients themselves are completely removed from the operation. For sum operation, there is no difference with skipping, but for average, dropping changes the divisor, resulting in a different outcome. In addition, we differentiate between *fine-grained* and *coarse-grained* skipping or dropping. Fine-grained refers to collective communication where each individual gradient is skipped (or dropped). For a coarse-grained implementation, gradients are coarsely skipped (or dropped) at slice-level granularity.

²The *ideal top- k* implementation will be discussed in Section 5.1

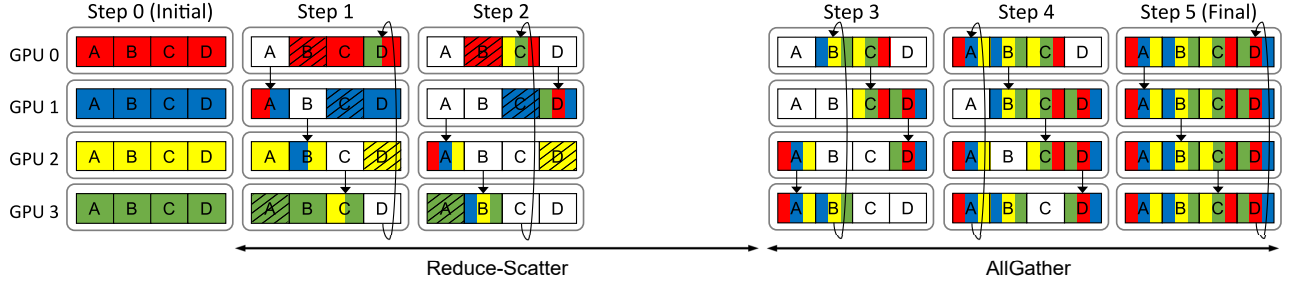


Figure 5: Proposed SkipReduce where one step in the Reduce-Scatter phase is skipped. Diagonally striped slices represent gradients that are effectively “dropped” during the Reduce-Scatter phase and are not used during the AllReduce collective communication. AllGather proceeds identically to the baseline AllReduce.

Algorithm 1 AllReduce with Fine-Grained Skipping

```

1: REDUCEORSKIP( $A, B, p, op$ ):
2:   for each index  $i$  in  $A$  do
3:     if  $\text{rand\_float}[0.0, 1.0] < p$  then
4:        $A[i] = B[i]$ 
5:     else
6:        $A[i] = op(A[i], B[i])$ 
7:   return  $A$ 

8: ALLREDUCE_DROPOUT( $p, op$ ):
9:   for  $i = 0$  to  $(N - 2)$  do //  $(N-1)$  Reduce-Scatter steps
10:     $\text{send\_slice} = (\text{rank} - i) \times N$ 
11:     $\text{recv\_slice} = (\text{rank} - i - 1) \times N$ 

12:     $\text{SEND}(\text{rank}, \text{send\_slice}, \text{rank} + 1)$ 
13:     $\text{recv\_data} = \text{RECEIVE}(\text{rank} - 1)$ 

14:     $T[\text{recv\_slice}] = \text{REDUCEORSKIP}(T[\text{recv\_slice}],$ 
15:       $\text{recv\_data}, p, op)$ 
16:  ALLGATHER() // same as baseline AllReduce

```

We first summarize different variations of AllReduce and SkipReduce collective communications that are presented in this work.

- **AllReduce_Dropout** : *fine-grained skipping* of individual gradients. Individual gradients are randomly skipped.
- **SkipReduce**: *coarse-grained gradient skipping* where some steps of AllReduce are skipped and gradients are effectively replaced by zeros.
- **DropReduce**: *coarse-grained gradient dropping* where some steps of AllReduce are skipped and gradients are completely removed by adjusting the divisor (for average operation).

4.2 SkipReduce Algorithm

Overview of AllReduce with fine-grained skipping, an approach similar to Dropout [45] but applied to gradients, referred to as AllReduce_Dropout, is shown in Algorithm 1. The algorithm is based

Algorithm 2 Static SkipReduce

```

1: SKIPREDUCE( $S, op$ ): //  $S$ : # of Reduce-Scatter steps to skip
2:   for  $i = S$  to  $(N - 2)$  do // Skip  $S$  Reduce-Scatter steps
3:      $\text{send\_slice} = (\text{rank} - i) \times N$ 
4:      $\text{recv\_slice} = (\text{rank} - i - 1) \times N$ 

5:      $\text{SEND}(\text{rank}, \text{send\_slice}, \text{rank} + 1)$ 
6:      $\text{recv\_data} = \text{RECEIVE}(\text{rank} - 1)$ 

7:      $T[\text{recv\_slice}] = \text{REDUCEORSKIP}(T[\text{recv\_slice}],$ 
8:        $\text{recv\_data}, 0, op)$ 
9:   ALLGATHER() // same as baseline AllReduce

```

on the conventional Ring AllReduce and the only difference is the introduction of a gradient skipping probability to the reduction kernel (line 14) – to determine whether the gradient should be used or “skipped.” For each element in the slice, a random decision is made based on the Bernoulli process to either reduce or skip the local gradient (line 3). If a gradient is skipped, the current GPU simply forwards the gradient that it receives from the previous GPU without adding the local gradient (line 4). This is equivalent to treating the local gradient as zero for that particular operation. If a gradient is not skipped, the current GPU reduces its own local gradient with the gradient received from the previous GPU, and forwards the result to the next GPU (line 6). This behavior is equivalent to the default reduction kernel in NCCL.

However, fine-grained skipping (AllReduce_Dropout) does not reduce the message size or the amount of data transmitted between GPUs, but merely reduces the information content that each message carries. For example, without skipping, a packet contains the reduction results from all $(N - 1)$ previous GPUs. In contrast, with skipping, a packet only contains results from $(N - 2)$ GPUs or fewer but the message size remains constant. On the other hand, SkipReduce leverages the fact that NCCL communicates gradients in slices (Section 2.2). By skipping AllReduce steps, it effectively skips gradient “slices”, not individual gradients, to reduce communication time. Figure 5 provides an example of SkipReduce where, by skipping one step from the Reduce-Scatter phase, an entire gradient slice from each GPU is skipped from the final reduction. This directly shortens the duration of the collective operation. Theoretically, skipping S steps of Reduce-Scatter cuts the communication

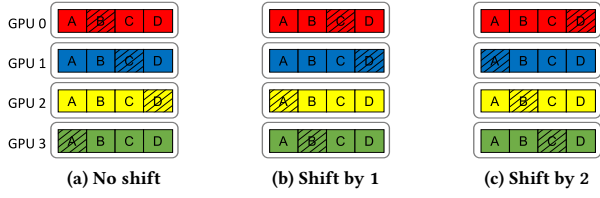


Figure 6: The skipped slices (shown by the diagonal stripes) in (a) baseline SkipReduce, (b) with the slice indices being shifted by one rank, and (c) shifted by two ranks.

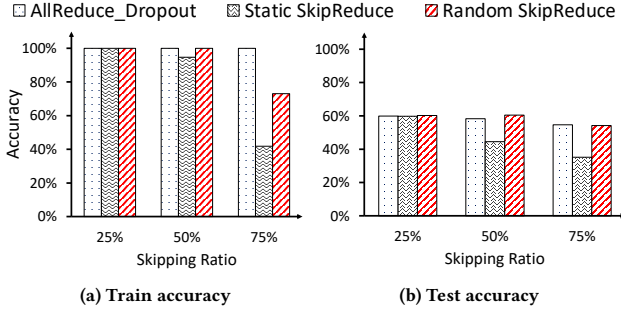


Figure 7: VGG-19 train and test accuracy with AllReduce_Dropout, Static SkipReduce and Random SkipReduce. Accuracy is normalized to baseline AllReduce with no skipping of gradients.

time by a fraction of $S/2(N-1)$ while skipping a corresponding S/N fraction of the gradients.

An overview of the SkipReduce algorithm is provided in Algorithm 2. SkipReduce is implemented by reducing the number of iterations in the Reduce-Scatter loop. If S is the number of Reduce-Scatter steps to be skipped, the Reduce-Scatter phase completes in $(N-1-S)$ steps (line 2), while the AllGather phase is unmodified and finishes in $(N-1)$ steps (line 8). As we show later in Section 5.2, steps in the AllGather can also be skipped but result in higher loss of accuracy. Since SkipReduce only skips slices (coarse-grained), the fine-grained gradient skipping probability (p) of the ReduceOrSkip function is set to zero (line 7) as this is only needed for fine-grain skipping.

4.3 Random SkipReduce

As discussed earlier, SkipReduce reduces communication time by skipping steps within AllReduce (e.g., Reduce-Scatter steps), skipping data in slice-level granularity in the process. However, naive implementation of SkipReduce (Section 4.2) is deterministic, e.g., each GPU skips the same slice during each iteration. This causes unfairness (or bias), as the same particular slices do not contribute to the collective communication result. To allow each GPU to skip different slices at each iteration, the slice indexes (Algorithm 2, line 3 and 4) can be shifted by a random offset that is regenerated every iteration. We refer to this as *Random SkipReduce*, as compared to *Static SkipReduce* in Section 4.2. This ensures that the skipped slices vary across iterations, promoting randomness in the skipped

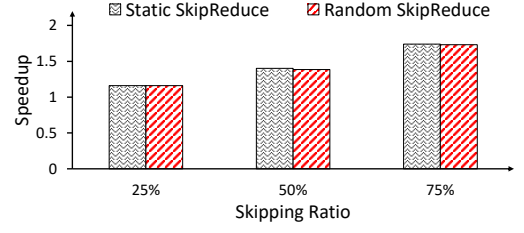


Figure 8: Speedup of Static and Random SkipReduce for different gradient skipping ratios, relative to AllReduce (no gradients skipped) using nccl-test with 1 GB message size AllReduce.

gradient slices. One key challenge is that all GPUs must generate the same random number to ensure that each GPU has a unique index. We implement this without additional synchronizations by modifying NCCL to pass the current iteration count from the host to all GPUs. This iteration count serves as a common seed, ensuring that the random number generator on each GPU is synchronized. An example is shown in Figure 6 where shifting the slice indices changes which slices are skipped by the end of the collective communication for a given training iteration. The skipped slices when no shifting is applied are exactly the same as the example shown in Figure 5.

Figure 7 compares the training accuracy when Static and Random SkipReduce are used for collective communication on VGG-19 training.³ AllReduce_Dropout skips gradients in element-wise granularity and thus provides no communication time reduction. Random SkipReduce achieves higher accuracy than Static SkipReduce as the skipping ratio increases, where Random SkipReduce improves the test accuracy by 19 percentage points at a 75% skipping rate. For all rates, Random SkipReduce matches the test accuracy achieved with AllReduce_Dropout. To understand the performance overhead of randomizing the slice indexes, Figure 8 compares the synthetic, collective communication-only performance of Random SkipReduce against Static SkipReduce. Static SkipReduce closely approaches the theoretical speedup of $\frac{2(N-1)}{2(N-1)-S}$, where N is the number of nodes and S is the number of Reduce-Scatter steps skipped. In comparison, Random SkipReduce achieves a speedup that is nearly identical to its static counterpart, with a performance (i.e., communication time) overhead of only up to 1.1%. This demonstrates that randomization, which improves model accuracy, can be implemented with negligible overhead.

4.4 Selective SkipReduce

The magnitude of a gradient is often used as a proxy for its “importance” [5]. For example, top- k sparsification techniques, which preserve the top- k largest gradients in an element-wise manner, have been shown to outperform random-based approaches [46]. In this section, we adapt this principle for SkipReduce, but at a coarser, layer-level granularity, rather than an element-wise one. This enables SkipReduce to skip gradients *selectively* to preserve model accuracy.

³Evaluation methodology is described in Section 5.1.

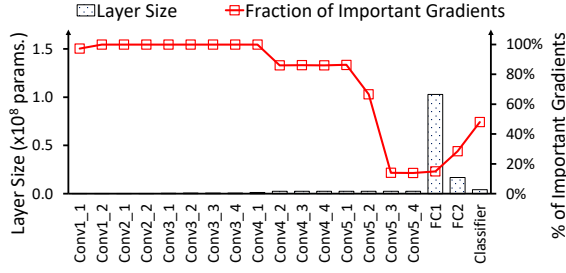


Figure 9: Layer size (number of parameters) and concentration of important gradients within each layer of VGG-19. An important gradient is defined as one that ranks among the global top-25% by magnitude.

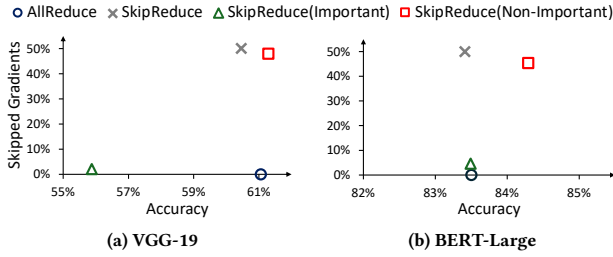


Figure 10: Tradeoff between the fraction of skipped gradients and test accuracy of (a) VGG-19 and (b) BERT-Large as different parts of the models (or layers) are skipped. SkipReduce skips across all layers, whereas SkipReduce() refers to skipping only the layers specified in the parentheses. The fraction of skipped gradients varies as different layers are targeted.

Figure 9 shows the layer size or the number of parameters for each layer of VGG-19, along with the distribution of important gradients throughout the model. The important gradients are those whose magnitudes are within the global top 25%, sampled at the end of the first epoch. This plot reveals that gradient sparsity is non-uniform across the layers of the model, as shown by the concentration of large-magnitude gradients in certain layers. Since the magnitude of a gradient correlates with its importance, layers with a high concentration of large-magnitude gradients can be considered more important and thus more sensitive to skipping. Conversely, layers with fewer important gradients are less sensitive and are better candidates for skipping. Furthermore, we observe that the important layers can be much smaller in size than the non-important ones. This presents a key opportunity to achieve significant communication speedup by skipping the large, non-important layers while protecting the small, critical ones to maintain training accuracy. While understanding why some layers have relatively higher importance may require rigorous theoretical understanding and is outside the scope of this work, the importance of certain layers can be intuitive. For instance, we can expect the first convolutional layer in VGG-19 and the embedding layer in transformers to be more sensitive to skipping as they initiate the projection of the input into the network’s latent space.

Table 2: Evaluated models and their training datasets.

Tasks	Models	Datasets
Image classification	VGG-19 [44]	CIFAR-100 [21]
Multiple choice	BERT-Large [10]	SWAG [56]
Sentence understanding	LLaMA-3.2 1B [16]	MNLI [54]

To further demonstrate the difference in sensitivity to skipping across different layers of a network, Figure 10 shows the test accuracy as we protect (not skip) some of the layers in the model. The ratio of skipped gradients is calculated by multiplying the proportion of the protected layers relative to the overall model size by a fixed skipping rate of 50%. Ideally, we aim to skip as many gradients as possible to achieve communication speedup while maintaining model accuracy. Therefore, data points closer to the top-right corner of the scatter plot are preferred. Figure 10a provides an example of the reduced benefit of protecting less important layers and protecting more gradients does not simply result in higher accuracy. By protecting the important convolutional layers whose number of parameters amounts to only 4% of the total model size, we are able to match and even improve the accuracy by 0.23 percentage point compared to AllReduce (no gradients are skipped). However, by protecting the fully connected layers instead, the accuracy drops by 5 percentage points even though the remaining 96% of the gradients are protected. A similar trend is observed in BERT-Large, as shown in Figure 10b. Protecting the important layers (i.e., embedding) leads to a significant improvement in accuracy. This observation motivates the design of SkipReduce to selectively skip gradients by protecting (not skipping) the important layers.

To enable selective gradient skipping in SkipReduce, we leverage the fact that gradients are not communicated all at once, but are instead partitioned into buckets. This design, where each bucket is processed by a separate AllReduce kernel launch, allows communication to be overlapped with computation. By exploiting this structure, we can apply different skipping rates to different buckets – aggressively skipping gradients in buckets associated with non-important layers, while preserving those in buckets containing important layers. For the rest of this work, SkipReduce refers to *Selective SkipReduce* unless otherwise specified – gradients at the non-important layers are skipped randomly, as in *Random SkipReduce*.

5 Evaluation

5.1 Methodology

SkipReduce is implemented within NCCL version 2.25.1 [1]. The cuRAND [2] library is used to generate the random numbers required to implement Random SkipReduce (Section 4.3). PyTorch [33] is then linked to the modified NCCL to implement SkipReduce for training. NCCL is configured to use Ring AllReduce algorithm. Evaluation is conducted on a system with 8 A40 GPUs, interconnected via PCIe 4.0. The models used for the evaluation and their training datasets are listed in Table 2. We train VGG-19 for 150 epochs using SGD optimizer with initial learning rate of 10^{-3} , momentum of 0.9, weight decay of 5×10^{-4} , and global batch size of 512. Both BERT-Large and LLaMA-3.2 use Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and initial learning rates of 2×10^{-5} for BERT-Large and 5×10^{-5}

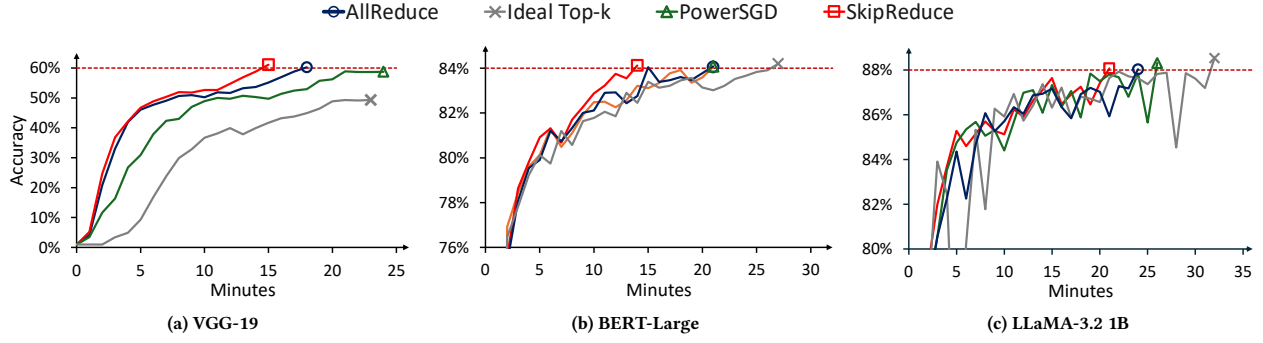


Figure 11: Time-to-accuracy of various collective communications for (a) VGG-19, (b) BERT-Large, and (c) LLaMA-3.2 1B. The red, horizontal dashed lines denote the target accuracy for each model.

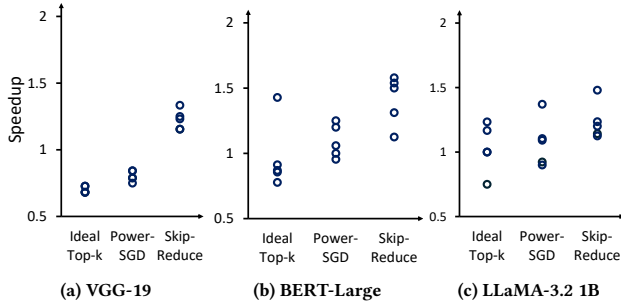


Figure 12: Time-to-accuracy speedup of various collective communications relative to baseline AllReduce across five runs. Each marker corresponds to one run.

for LLaMA-3.2. We use global batch sizes of 16 and 64 for BERT-Large and LLaMA-3.2, respectively. For all models, the parameters and gradients are represented using FP32. Data parallelism is leveraged to train all the models using the DistributedDataParallel provided by PyTorch [33].

We compare SkipReduce against baseline AllReduce, ideal top- k , and PowerSGD. Unless otherwise stated, we configure top- k to select 1% of the gradients, PowerSGD with a rank of 4, and SkipReduce to skip 50% of the Reduce-Scatter steps. For all collective communications, communication is overlapped with computation whenever possible. To the best of our knowledge, prior works on top- k gradient compression have only been implemented using MPI-based communication libraries [25, 30, 38, 42]. Therefore, comparing these approaches against SkipReduce, implemented on NCCL, is not a fair comparison. To address this, we implement an *idealized* version of top- k , in which the indices and values of the selected gradients are communicated as if they were dense tensors. This eliminates the fill-in issue as no sparse reductions are used and provides a *lower bound* on the communication time for top- k approaches, assuming they could be efficiently implemented in NCCL. However, this implementation is not functionally correct as it does not support actual sparse reductions. Therefore, to measure its time-to-accuracy (TTA), we adopted a different approach. We

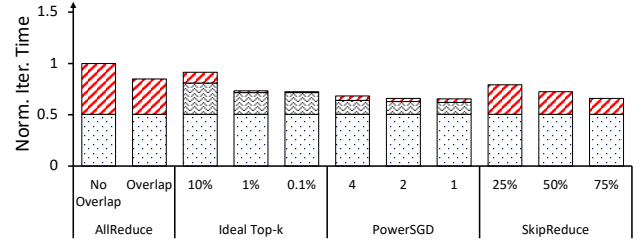


Figure 13: Breakdown of LLaMA-3.2 1B iteration time for various collective communications. The iteration time is normalized to AllReduce (no overlapping).

first ran the baseline AllReduce but zeroed out all non-selected gradients before communication to simulate the behaviour of top- k . We then evaluated the model every N iterations, where N corresponds to the number of iterations that the ideal top- k implementation could execute within a single sampling period. For PowerSGD, we perform a warm-up for the first 1000 iterations, where gradients are communicated using baseline AllReduce. We apply error correction mechanisms for both top- k and PowerSGD: top- k uses SGD with memory [46], while PowerSGD uses error-feedback SGD [51]. These mechanisms require additional memory equal to the size of the model. In contrast, SkipReduce does not employ any error correction strategy and therefore maintains a smaller memory footprint.

5.2 Results

We conducted time-to-accuracy (TTA) measurements for various collective communications across different models over five runs with unique random seeds. In Figure 11, we present the runs where SkipReduce demonstrates the median speedup relative to baseline AllReduce. The distribution of TTA speedups across all runs is shown as a dot plot in Figure 12. For the results shown in Figure 12a, we configured top- k to select 5% of the gradients (up from 1%) to reach the target accuracy at the expense of increased iteration time. SkipReduce is able to outperform baseline AllReduce and prior works in all workloads and runs. However, it is interesting to note that SkipReduce does not always yield faster iteration times

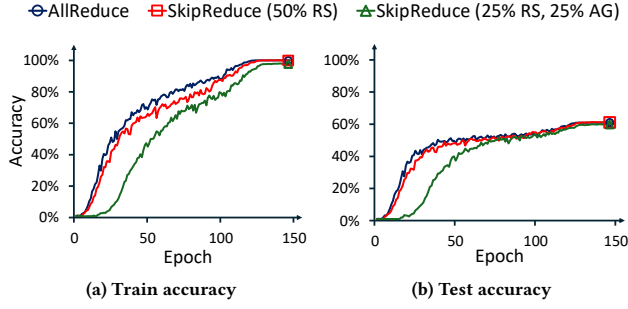


Figure 14: VGG-19 train and test accuracy when SkipReduce skips 50% of the Reduce-Scatter (RS) steps, compared to when 25% of both RS and AllGather (AG) steps are skipped.

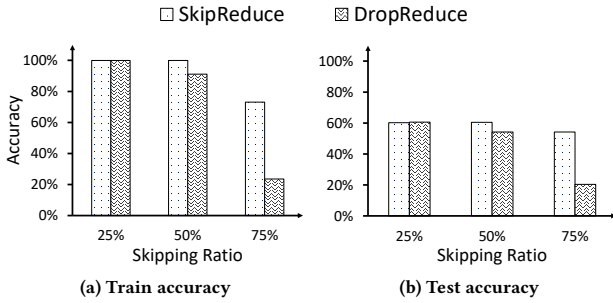


Figure 15: VGG-19 train and test accuracy with SkipReduce and DropReduce.

compared to ideal top- k and PowerSGD. For instance, on LLaMA-3.2, SkipReduce exhibits a 6% slower iteration time than PowerSGD, as shown in Figure 13. Despite this, SkipReduce achieves an average TTA speedup of 16% relative to PowerSGD across five runs. This is because SkipReduce does not require aggressive sparsification or compression, thus retaining a significant amount of information while achieving comparable iteration time to PowerSGD, which highlights the limitations of prior works in higher bandwidth settings as discussed in Section 3.

Another design consideration is to extend SkipReduce to also skip the AllGather steps, since the current implementation reduces communication time by only targeting the Reduce-Scatter phase. While the communication time can be further cut down by skipping the AllGather steps, the performance tradeoff when skipping AllGather steps is more prominent compared to skipping the Reduce-Scatter steps. Figure 14 shows the performance of SkipReduce when 50% of the gradients are skipped at the Reduce-Scatter phase, compared to when 25% of gradients are skipped at both Reduce-Scatter and AllGather. Since the total number of steps skipped is the same, both schemes achieve similar communication speedup. However, skipping the AllGather steps significantly slows down convergence and reduces the final train and test accuracy by 1.95 and 1.3 percentage points, respectively. Skipping AllGather steps is more costly as the skipped gradients carry reduced information from all other GPUs. Thus, skipping the AllGather steps causes higher information loss relative to Reduce-Scatter. Moreover, skipping the AllGather phase causes the model replicas on each GPU to diverge, violating

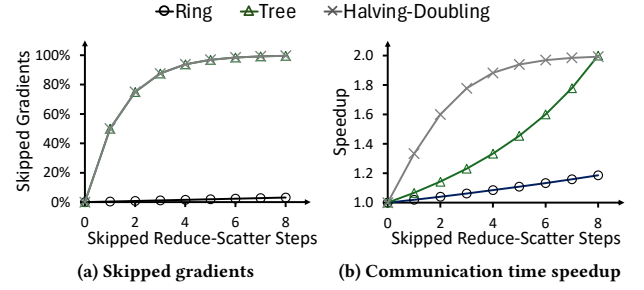


Figure 16: Ratio of skipped gradients and the communication time speedup achieved for each skipped Reduce-Scatter step for various AllReduce logical topologies with 256 nodes.

the synchronization requirements of data parallel training. Consequently, for our evaluation, we measure accuracy using only one sample of the replicas.

In data parallel training, the model weights are updated using the average of the local gradients across all GPUs. For SkipReduce, the skipped gradients are effectively replaced with a value of zero as SkipReduce divides the gradients by the number of GPUs (N), regardless of how many steps are skipped. An alternative design, however, would be to adjust this divisor to the number of remaining Reduce-Scatter steps ($N - S$), effectively removing the gradients. We refer to this as DropReduce (Section 4.1) and compare it with SkipReduce in Figure 15. However, we observed no benefit and, in fact, saw performance degradation. One explanation is that by completely removing the gradients, the contribution of the remaining gradients is amplified, possibly creating bias. In addition, as shown earlier, gradients often have a distribution near 0 – thus, replacing the skipped gradients with zeros is a reasonable approximation. An open question is whether substituting the skipped gradients with other non-zero values can help improve accuracy.

6 Discussion

In this section, we provide additional analysis on different implementations of SkipReduce, including its implications on other logical topologies, its usage across other types of parallelism, and its benefits as a form of regularization.

6.1 Alternative Logical Topologies

In this work, we focus on the Ring-based AllReduce algorithm where the communication pattern (or the logical topology [8]) is a ring since it is commonly used and has been shown to be bandwidth optimal [34]. However, SkipReduce can also be applied to other logical topologies, each with varying scaling properties. Each logical topology offers a different balance between the amount of skipped gradients (which affects training accuracy) and the amount of communication time saved.

An analytical evaluation of SkipReduce on various logical topologies with 256 nodes is presented in Figure 16. For ring and halving-doubling topologies, the communication time speedup and the fraction of skipped gradients scale in a similar manner – linearly for ring and logarithmically for halving-doubling. This means that the trade-off between training speed and accuracy loss remains

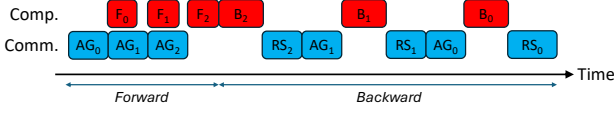


Figure 17: Communications in sharded data parallel training. For layer i , F_i/B_i : forward/backward computation, AG_i : AllGather of model weights, RS_i : Reduce-Scatter for gradients.

consistent as more communication steps are skipped. In contrast, there is a mismatch in the scaling properties of both metrics on a logical tree topology. For a tree, the ratio of skipped gradients scales logarithmically, while the communication time speedup scales approximately linearly. This suggests that for a tree, skipping the first few steps yields a small speedup at a high cost to accuracy. However, this tradeoff becomes increasingly favorable as more communication steps are skipped.

6.2 Training Convergence

Prior works [5, 46] have provided convergence guarantees of stochastic gradient descent (SGD) with gradient compression operators. Given a total gradient size of d , the compression operator compresses the gradient size to k , where $0 < k \leq d$. SkipReduce is essentially a *random* gradient compressor, whose convergence is also proven [46]. However, one essential difference between SkipReduce and prior works [3, 25, 30, 38, 41, 42] is that these previous methods use additional memory space to accumulate gradients that are skipped in the current iteration, to possibly be sent in the next iterations, while SkipReduce “forgets” all the skipped gradients. Without gradient accumulation, a random gradient compressor theoretically requires at worst d/k times more iterations to achieve the same model quality as baseline SGD or other gradient compression schemes with gradient accumulation support [46]. However, our empirical results show that SkipReduce can match and even exceed the accuracy of baseline AllReduce on some DNN models with the same amount of training epochs. The intuition here is that since SkipReduce does not simply skip gradients randomly but also selectively, the gradients it skips are typically close to 0. Therefore, skipping them has a minimal impact on training quality, provided the skipping or dropping ratio (k) is reasonably small.

6.3 Sharded Data Parallel (SDP)

While data parallel (DP) training accelerates distributed training, the scalability is limited to models that can fit within a single GPU, as it requires replicating the model across multiple GPUs. To overcome this limitation, Sharded Data Parallelism (SDP) [35, 57] partitions or *shards* not just the input data, but also the model parameters, optimizer states, and gradients across the GPUs. This results in a reduction in memory usage and enabling larger models to be trained [35, 57].⁴ A high-level overview of SDP training across a hypothetical 3-layer network is shown in Figure 17, detailing the sequence of computation and communication across the three layers.

⁴Sharding used in SDP is different from tensor and pipeline parallelism, where the models are always sharded throughout the entire training. While tensor and pipeline parallelism provide further memory savings, they require additional communication to communicate the activation values.

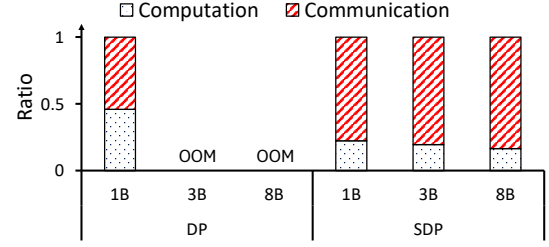


Figure 18: Communication overhead of LLaMA-3 training in an 8-GPU system across various model sizes for data parallel (DP) and sharded data parallel (SDP) training. OOM: out-of-memory error.

Each GPU is assigned a model shard. During the forward pass, an AllGather (AG_i) operation is required to gather the parameters before computing the activation for layer i . After computation, these gathered shards are discarded to save memory.⁵ The backward pass follows a similar pattern. First, an AllGather is performed to collect the parameters needed for gradient computation. This is followed by a Reduce-Scatter (RS_i) operation where each GPU receives the final reduced gradient for its assigned shard.⁶

SDP presents new challenges and opportunities with SkipReduce. It trades reduced memory usage for higher communication overhead, resulting in a higher amount of exposed communication (Figure 18). Furthermore, the communication pattern is not necessarily AllReduce, as the AllGather and Reduce-Scatter operations are performed on different data (weights and gradients, respectively). Accelerating either the Reduce-Scatter or AllGather communication individually is straightforward – SkipReduce can be applied identically to how SkipReduce was implemented in DP (Figure 5). However, accelerating *both* Reduce-Scatter and AllGather communications simultaneously requires a key modification. To minimize information loss, SkipReduce must ensure that the skipped gradients correspond precisely to the weights omitted during the AllGather phase. This alignment minimizes information loss by exclusively gathering gradients for weights actively participating in training, and is achieved by running the Reduce-Scatter in the *opposite direction* of the AllGather. However, by default, NCCL executes ring-based collectives in the same direction for each logical ring. Thus, we modified NCCL to reverse the sender-receiver pairs during Reduce-Scatter, enabling the two operations to run in opposing directions on the same logical ring.

When an AllGather step is skipped in SDP, each GPU effectively drops the connections associated with the missing parameter shard, assuming its weights are zero. Consequently, each GPU trains a unique subnetwork, creating an ensemble effect across the system. Note that in traditional (DP) training, each GPU holds a full, identical replica of the model. In DP, skipping AllGather steps would compromise model consistency since each GPU will receive different reduced slices, causing the model replicas across the GPUs to diverge. However, this problem does not arise in SDP, where each

⁵We assume that the parameters, gradients, and optimizer states are sharded, which is equivalent to FSDP [57] with full sharding strategy and ZeRO-3 [35].

⁶Note AG_2 is not shown before B_2 as it reuses shards gathered before executing F_2 .

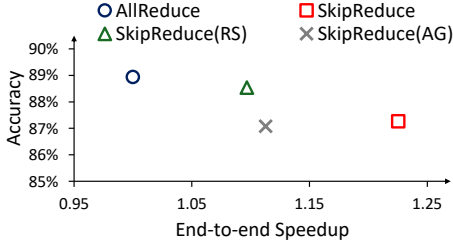


Figure 19: Tradeoff between performance (i.e., iteration time) speedup and accuracy of LLaMA 3.2 1B using sharded data parallel training. SkipReduce(RS) and SkipReduce(AG) indicate that SkipReduce is only implemented to the Reduce-Scatter or AllGather communication, respectively.

GPU only maintains its assigned model shard, not a full replica. This distinction becomes critical during inference. The full model can be reconstructed in SDP by simply gathering the shards. In DP, however, skipping AllGather steps would result in multiple, different final models, making it unclear which version to use for inference.

Figure 19 shows the trade-off between test accuracy and end-to-end speedup of SkipReduce implemented in PyTorch FSDP [57] when training the LLaMA 3.2 model. Applying SkipReduce only to the gradients communicated using Reduce-Scatter communication yields a 9% speedup, while extending SkipReduce to also skip the weights communicated in AllGather doubles the speedup to 22%, at the cost of a slight reduction in accuracy. In our evaluation, we configured SkipReduce to skip 50% of the weights and gradients. However, given the criticality of model parameters, one interesting modification to SkipReduce was the need to *warm-up* the model, i.e., initially use baseline AllGather (without skipping any steps) for a certain period before applying SkipReduce. This ensures that the training converges and the accuracy loss is minimized. Interestingly, skipping only the AllGather communication phase results in the lowest accuracy. This suggests that preserving the gradients of the weights that do not contribute in the iteration (due to being skipped in the AllGather) may actually be more harmful to the model’s performance than simply ignoring them would be.

6.4 Implications on Regularization

Dropout [45] regularizes a model by dropping intermediate activation values, effectively creating a different subnetwork at each iteration. On the other hand, SkipReduce does not modify the network connections but instead introduces noise during weight updates by skipping or nullifying a fraction of the gradients, an approach similar to [49]. Compared to Dropout, SkipReduce operates at a coarser granularity to reduce communication time. Prior works [13, 22] have considered coarse-grained Dropout but mainly focus on model performance.

The regularization benefit of SkipReduce is demonstrated in Figure 10, where, by protecting the sensitive layers and skipping gradients selectively, SkipReduce achieves higher test accuracy than baseline AllReduce, even while skipping gradients to accelerate communication. Figure 20 further compares the regularization benefits provided by Dropout versus the inherent regularizing effect

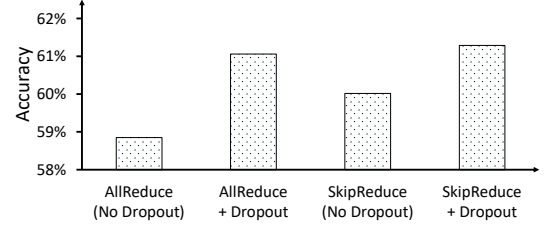


Figure 20: VGG-19 test accuracy for baseline AllReduce and SkipReduce, with and without Dropout.

of SkipReduce. The SkipReduce skipping rate is configured to match the Dropout rate used in VGG, which is 50% [44]. Dropout provides better regularization compared to SkipReduce, with an additional 1% and 0.3% increase in test accuracy for Dropout and SkipReduce, respectively. However, when SkipReduce and Dropout are used together, we see a cumulative benefit, providing additional 1.2% increase in test accuracy. This demonstrates that the two methods are not only compatible, but also complementary.

7 Conclusion

In this work, we show how gradients (and packets) can be skipped during AllReduce collective communication of data parallel training to improve communication performance. By reducing the number of steps required for the AllReduce operation, effectively skipping the gradients, the communication acceleration is achieved with minimal computational and memory overhead. We evaluate SkipReduce against prior works and demonstrate that the computational overhead of compression techniques used in prior works can hinder their effectiveness on channel bandwidths typically available in systems with commodity GPUs and in large-scale high-performance clusters. To preserve model accuracy, we first introduce a randomized version of SkipReduce that can be implemented with minimal overhead. We further enhance SkipReduce by observing that gradient sparsity is non-uniform across model layers, and enable SkipReduce to selectively skip gradients. Our results show that SkipReduce provides speedup in end-to-end training time while preserving accuracy. More importantly, this work introduces how SkipReduce principles can go beyond simply accelerating AllReduce collective communication – we explore its impact as a regularizer, its applicability to other logical topologies, and its adoption for other types of parallelism.

Acknowledgments

We would like to thank the anonymous reviewers for their comments. This work is supported in part by Samsung project G01240469, IITP grants funded by MSIT (no. RS-2023-00228255, RS-2024-00402898, RS-2025-02214654, RS-2025-02217106), in part by NRF (no. RS-2023-NR077034), and in part by Hyundai Motor Chung Mong-Koo Global Scholarship.

References

- [1] 2023. Nvidia Collective Communication Library (NCCL). <https://developer.nvidia.com/nccl>. Accessed: 2023-07-23.
- [2] 2023. Nvidia CUDA Random Number Generation (cuRAND) library. <https://developer.nvidia.com/curand>. Accessed: 2023-07-23.

- [3] Alham Fikri Aji and Kenneth Heafield. 2017. Sparse Communication for Distributed Gradient Descent. *arXiv abs/1704.05021* (2017). [arXiv:1704.05021](http://arxiv.org/abs/1704.05021)
- [4] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. 2017. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc.
- [5] Dan Alistarh, Torsten Hoefler, Mikael Johansson, Sarit Khirirat, Nikola Konstantinov, and Cédric Renggli. 2018. The Convergence of Sparsified Gradient Methods. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (Montréal, Canada) (NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 5977–5987.
- [6] Tal Ben-Nun and Torsten Hoefler. 2019. Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis. *ACM Comput. Surv.* 52, 4, Article 65 (Aug. 2019), 43 pages. <https://doi.org/10.1145/3320060>
- [7] J. Bernauer and P. Kashinkunti. 2021. *Accelerating AI at-scale with Selene DGXA100 SuperPOD and Lustre Parallel Filesystem Storage*. Technical Report. NVIDIA, USA.
- [8] Sanghun Cho, Hyeonjun Son, and John Kim. 2023. Logical/Physical Topology-Aware Collective Communication in Deep Learning Training. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. Montreal, QC, Canada, 56–68. <https://doi.org/10.1109/HPCA56546.2023.10071117>
- [9] Tim Dettmers. 2016. 8-Bit Approximations for Parallelism in Deep Learning. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2–4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1511.04561>
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv abs/1810.04805* (2018). [arXiv:1810.04805](http://arxiv.org/abs/1810.04805)
- [11] Nikolai Dryden, Tim Moon, Sam Ade Jacobs, and Brian Van Essen. 2016. Communication Quantization for Data-Parallel Training of Deep Neural Networks. In *2nd Workshop on Machine Learning in HPC Environments (MLHPC)*. 1–8. <https://doi.org/10.1109/MLHPC.2016.004>
- [12] Jiawei Fei, Chen-Yu Ho, Atal N. Sahu, Marco Canini, and Amedeo Sapia. 2021. Efficient Sparse Collective Communication and Its Application to Accelerate Distributed Deep Learning. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (Virtual Event, USA) (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 676–691. <https://doi.org/10.1145/3452296.3472904>
- [13] Golnaz Ghiasi, Tsung-Yi Lin, and Quoc V. Le. 2018. DropBlock: a regularization method for convolutional networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (Montréal, Canada) (NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 10750–10760.
- [14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- [15] Richard L. Graham, Devendar Bureddy, Pak Lui, Hal Rosenstock, Gilad Shainer, Gil Bloch, Dror Goldener, Mike Dubman, Sasha Kotchubievsky, Vladimir Koushnir, Lion Levi, Alex Margolin, Tamir Ronen, Alexander Shpiner, Oded Wertheim, and Eitan Zahavi. 2016. Scalable Hierarchical Aggregation Protocol (SHaRP): A Hardware Architecture for Efficient Data Reduction. In *2016 First International Workshop on Communication Optimizations in HPC (COMHPC)*. 1–10. <https://doi.org/10.1109/COMHPC.2016.006>
- [16] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and et al. Arun Rao. 2024. The Llama 3 Herd of Models. [arXiv:2407.21783](https://arxiv.org/abs/2407.21783) [cs.AI]
- [17] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37 (Lille, France) (ICML'15)*. 1737–1746.
- [18] Jiayi Huang, Pritam Majumder, Sungkeun Kim, Abdullah Muzahid, Ki Hwan Yum, and Eun Jung Kim. 2021. Communication Algorithm-Architecture Co-Design for Distributed Deep Learning. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA '21)*. 181–194. <https://doi.org/10.1109/ISCA52012.2021.00023>
- [19] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (Orlando, FL, USA) (ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. <https://doi.org/10.1145/3579371.3589350>
- [20] Benjamin Klenk, Nan Jiang, Greg Thorson, and Larry Dennison. 2020. An In-Network Architecture for Accelerating Shared-Memory Multiprocessor Collectives. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 996–1009. <https://doi.org/10.1109/ISCA45697.2020.00085>
- [21] Alex Krizhevsky. 2009. Learning Multiple Layers of Features from Tiny Images. <https://api.semanticscholar.org/CorpusID:18268744>
- [22] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. 2017. FractalNet: Ultra-Deep Neural Networks without Residuals. In *International Conference on Learning Representations (Toulon, France) (ICLR '17)*.
- [23] Sabuj Laskar, Pranati Majhi, Sungkeun Kim, Farabi Mahmud, Abdullah Muza-hid, and Eun Jung Kim. 2024. Enhancing Collective Communication in MCM Accelerators for Deep Learning Training. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 1–16. <https://doi.org/10.1109/HPCA57654.2024.00069>
- [24] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [25] Shigang Li and Torsten Hoefler. 2022. Near-Optimal Sparse Allreduce for Distributed Deep Learning. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (Seoul, Republic of Korea) (PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 135–149. <https://doi.org/10.1145/3503221.3508399>
- [26] Youjie Li, Iou-Jen Liu, Yifan Yuan, Deming Chen, Alexander Schwing, and Jian Huang. 2019. Accelerating Distributed Reinforcement learning with In-Switch Computing. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. 279–291.
- [27] Youjie Li, Jongse Park, Mohammad Alian, Yifan Yuan, Zheng Qu, Peitian Pan, Ren Wang, Alexander Schwing, Hadi Esmaeilzadeh, and Nam Sung Kim. 2018. A Network-Centric Hardware/Algorithm Co-Design to Accelerate Distributed Training of Deep Neural Networks. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 175–188. <https://doi.org/10.1109/MICRO.2018.00023>
- [28] Dongkyun Lim and John Kim. 2025. TidalMesh: Topology-Driven AllReduce Collective Communication for Mesh Topology. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1526–1540. <https://doi.org/10.1109/HPCA61900.2025.00114>
- [29] Hwijoon Lim, Juncheol Ye, Sangeetha Abdu Jyothi, and Dongsu Han. 2024. Accelerating Model Training in Multi-cluster Environments with Consumer-grade GPUs. In *Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 707–720. <https://doi.org/10.1145/3651890.3672228>
- [30] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and Bill Dally. 2018. Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training. In *International Conference on Learning Representations*.
- [31] Paulius Micikevicius. 2020. Fundamentals of scaling out DL training. *HotChips* (2020).
- [32] Deepak Narayanan, Mohammad Shoeibi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (St. Louis, Missouri) (SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 58, 15 pages. <https://doi.org/10.1145/3458817.3476209>
- [33] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc.
- [34] Pitch Patarasuk and Xin Yuan. 2009. Bandwidth optimal all-reduce algorithms for clusters of workstations. *J. Parallel and Distrib. Comput.* 69, 2 (2009), 117–124. <https://doi.org/10.1016/j.jpdc.2008.09.002>
- [35] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. [arXiv:1910.02054](https://arxiv.org/abs/1910.02054) [cs.LG] <https://arxiv.org/abs/1910.02054>
- [36] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. 2021. Enabling compute-communication overlap in distributed deep learning training platforms. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (Virtual Event, Spain) (ISCA '21)*. IEEE Press, 540–553. <https://doi.org/10.1109/ISCA52012.2021.00049>
- [37] Saeed Rashidi, William Won, Sudarshan Srinivasan, Srinivas Sridharan, and Tushar Krishna. 2022. Themis: a network bandwidth-aware collective scheduling policy for distributed training of DL models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (New York, New York) (ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 581–596. <https://doi.org/10.1145/3470496.3527382>
- [38] Cedric Renggli, Saleh Ashkboos, Mehdi Aghagholzadeh, Dan Alistarh, and Torsten Hoefler. 2019. SparCML: High-Performance Sparse Communication for Machine Learning. In *Proceedings of the International Conference for High Performance*

- Computing, Networking, Storage and Analysis (Denver, Colorado) (SC '19). Association for Computing Machinery, New York, NY, USA, Article 11, 15 pages. <https://doi.org/10.1145/3295500.3356222>
- [39] Christopher De Sa, Ce Zhang, Kunle Olukotun, and Christopher Ré. 2015. Taming the wild: a unified analysis of HOG WILD! -style algorithms. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2* (Montreal, Canada) (NIPS'15). MIT Press, Cambridge, MA, USA, 2674–2682.
- [40] Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 2014. 1-Bit Stochastic Gradient Descent and Application to Data-Parallel Distributed Training of Speech DNNs. In *Interspeech 2014* (interspeech 2014 ed.).
- [41] Shaohuai Shi, Xiaowen Chu, Ka Chun Cheung, and Simon See. 2019. Understanding Top-k Sparsification in Distributed Deep Learning. *arXiv abs/1911.08772* (2019). [arXiv:1911.08772](https://arxiv.org/abs/1911.08772) <https://arxiv.org/abs/1911.08772>
- [42] Shaohuai Shi, Qiang Wang, Kaiyong Zhao, Zhenheng Tang, Yuxin Wang, Xiang Huang, and Xiaowen Chu. 2019. A Distributed Synchronous SGD Algorithm with Global Top-k Sparsification for Low Bandwidth Networks. *arXiv abs/1901.04359* (2019). [arXiv:1901.04359](https://arxiv.org/abs/1901.04359) <https://arxiv.org/abs/1901.04359>
- [43] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv abs/1909.08053* (2019). [arXiv:1909.08053](https://arxiv.org/abs/1909.08053) <https://arxiv.org/abs/1909.08053>
- [44] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *3rd International Conference on Learning Representations (ICLR '15)*. San Diego, CA, USA.
- [45] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 15, 56 (2014), 1929–1958.
- [46] Sebastian U. Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. 2018. Sparsified SGD with Memory (NIPS'18). Curran Associates Inc., Red Hook, NY, USA, 4452–4463.
- [47] Nikko Strom. 2015. Scalable distributed DNN training using commodity GPU cloud computing. In *Proc. Interspeech 2015*. 1488–1492. <https://doi.org/10.21437/Interspeech.2015-354>
- [48] Emil Talpes, Douglas Williams, and Debjit Das Sarma. 2022. DOJO: The Microarchitecture of Tesla's Exa-Scale Computer. In *IEEE Hot Chips 34 Symposium (HCS '22)*. 1–28. <https://doi.org/10.1109/HCS55958.2022.9895534>
- [49] Hung-Yu Tseng, Yi-Wen Chen, Yi-Hsuan Tsai, Sifei Liu, Yen-Yu Lin, and Ming-Hsuan Yang. 2020. Regularizing Meta-Learning via Gradient Dropout. In *Asian Conference on Computer Vision (Kyoto, Japan) (ACCV '20)*.
- [50] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Commun. ACM* 33, 8 (Aug. 1990), 103–111. <https://doi.org/10.1145/79173.79181>
- [51] Thijs Vogels, Sai Praneeth Karimireddy, and Martin Jaggi. 2019. PowerSGD: practical low-rank gradient compression for distributed optimization. Curran Associates Inc., Red Hook, NY, USA.
- [52] Hao Wang, Han Tian, Jingrong Chen, Xinchun Wan, Jiacheng Xia, Gaoxiong Zeng, Wei Bai, Junchen Jiang, Yong Wang, and Kai Chen. 2024. Towards Domain-Specific Network Transport for Distributed DNN Training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 1421–1443. <https://www.usenix.org/conference/nsdi24/presentation/wang-hao>
- [53] Ertza Warraich, Omer Shabtai, Khalid Manaa, Shay Vargaftik, Yonatan Piasetzky, Matty Kadosh, Lalith Suresh, and Muhammad Shahbaz. 2025. OptiReduce: Resilient and Tail-Optimal AllReduce for Distributed Deep Learning in the Cloud. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 685–703. <https://www.usenix.org/conference/nsdi25/presentation/warraich>
- [54] Adina Williams, Nikita Nangia, and Samuel R. Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, New Orleans, Louisiana, 1112–1122. <https://aclanthology.org/N18-1101>
- [55] William Won, Midhlesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. 2024. TACOS: Topology-Aware Collective Algorithm Synthesizer for Distributed Machine Learning. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 856–870. <https://doi.org/10.1109/MICRO61859.2024.00068>
- [56] Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. SWAG: A Large-Scale Adversarial Dataset for Grounded Commonsense Inference. *arXiv:1808.05326* [cs.CL] <https://arxiv.org/abs/1808.05326>
- [57] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Mye Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *arXiv:2304.11277* [cs.DC] <https://arxiv.org/abs/2304.11277>

A Artifact Appendix

A.1 Abstract

This artifact provides the open-source implementation of SkipReduce. Step-by-step guide is provided to reproduce the results of SkipReduce and prior works, as reported in Figure 11.

A.2 Artifact check-list (meta-information)

- **Algorithm:** SkipReduce collective communication.
- **Program:** PyTorch.
- **Model:** VGG-19, BERT-Large, LLaMA-3.2 1B.
- **Data set:** CIFAR-100, SWAG, MNLI.
- **Run-time environment:** torch.
- **Hardware:** a multi-GPU system. Each GPU should have at least 40 GB of memory.
- **Metrics:** iteration time, accuracy.
- **Output:** txt files.
- **How much disk space required (approximately)?:** less than 10 GB.
- **How much time is needed to prepare workflow (approximately)?:** Less than 30 minutes.
- **How much time is needed to complete experiments (approximately)?:** Less than 6 hours to run across all evaluated collective communication algorithms and models, for a single random seed.

A.3 Description

A.3.1 How to access. We modified the HuggingFace transformers library to run evaluation for every time period. It is available at https://github.com/hanskasan/transformers_SkipReduce.git.

The modified NCCL can be downloaded from <https://github.com/hanskasan/SkipReduce-2.0.git>.

The scripts to run the evaluations are available at https://github.com/hanskasan/SkipReduce_MICRO2025.git.

The artifact is also available on Zenodo and can be accessed at <https://doi.org/10.5281/zenodo.16992079>.

A.3.2 Hardware dependencies. A multi-GPU system. For our evaluations, we rented NVIDIA A40 GPUs from RunPod. We rented pods with CUDA v.12.4. GPUs with at least 40 GB of memory are required.

A.3.3 Software dependencies. We ran our experiments on Python 3.11 and PyTorch 2.8.0. We also used additional packages from HuggingFace: transformers, datasets, evaluate, accelerate. To implement the modified NCCL, it has to be built from scratch.

A.3.4 Data sets. CIFAR-100, SWAG, MNLI. All accessible from HuggingFace datasets. A HuggingFace account may be required to access the datasets.

A.3.5 Models. VGG-19, BERT-Large, LLaMA-3.2 1B. All accessible from HuggingFace transformers. A HuggingFace account may be required to access the models. Access to some of these repositories is granted upon request. The request can be submitted to the HuggingFace page of the respective models.

A.4 Installation

These steps can be executed from any directory on the system. We assume a directory called PATH.

Editable install for HuggingFace Transformers.

- (1) Clone from https://github.com/hanskasan/transformers_SkipReduce.git.
- (2) Run `cd PATH/transformers_SkipReduce/`.
- (3) Run `pip install -e`.

Install the modified NCCL.

- (1) Clone from <https://github.com/hanskasan/SkipReduce-2.0.git>.
- (2) Run `cd PATH/SkipReduce-2.0/`.
- (3) Run `./make.sh`.

Configure the NCCL path.

- (1) Open `PATH/SkipReduce_MICRO2025/VGG/run_aot_vgg19.sh`.
- (2) Assign `LD_PRELOAD` to where `libnccl.so` of the modified NCCL is.
- (3) Repeat the first two steps for `PATH/SkipReduce_MICRO2025/BERT/run_aot_bert.sh`.
- (4) Repeat the first two steps for `PATH/SkipReduce_MICRO2025/Llama/run_aot_llama.sh`.

A.5 Experiment workflow

Initialization.

- (1) Clone from https://github.com/hanskasan/SkipReduce_MICRO2025.git.
- (2) Run `cd PATH/SkipReduce_MICRO2025/`.
- (3) Open `initialize.sh` and replace `<YOUR_TOKEN>` with your HuggingFace token.
- (4) Run `./initialize.sh`.

Run VGG evaluations.

- (1) Run `cd PATH/VGG/`.
- (2) Run `mkdir aot_reports/`.
- (3) Run `./run_aot_vgg19.sh`.
- (4) All the results will be available in `aot_reports/`.

Run BERT-Large evaluations.

- (1) Run `cd PATH/BERT/`.
- (2) Run `mkdir aot_reports/`.
- (3) Run `./run_aot_bert.sh`.
- (4) All the results will be available in `aot_reports/`.

Run LLaMA-3.2 evaluations.

- (1) Run `cd PATH/Llama/`.
- (2) Run `mkdir aot_reports/`.
- (3) Run `./run_aot_llama.sh`.
- (4) All the results will be available in `aot_reports/`.

A.6 Evaluation and expected results

Users are expected to observe the same trend as in Figure 11, where SkipReduce is able to achieve target accuracy faster than prior works. Slight variance in how the accuracy progresses over time is expected due to the non-deterministic nature of the workload, but should not affect the trends reported in the paper.

A.7 Experiment customization

The provided scripts allow users to easily modify the training hyperparameters, e.g., random seed, learning rate, batch size.

A.8 Notes

None.

A.9 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>